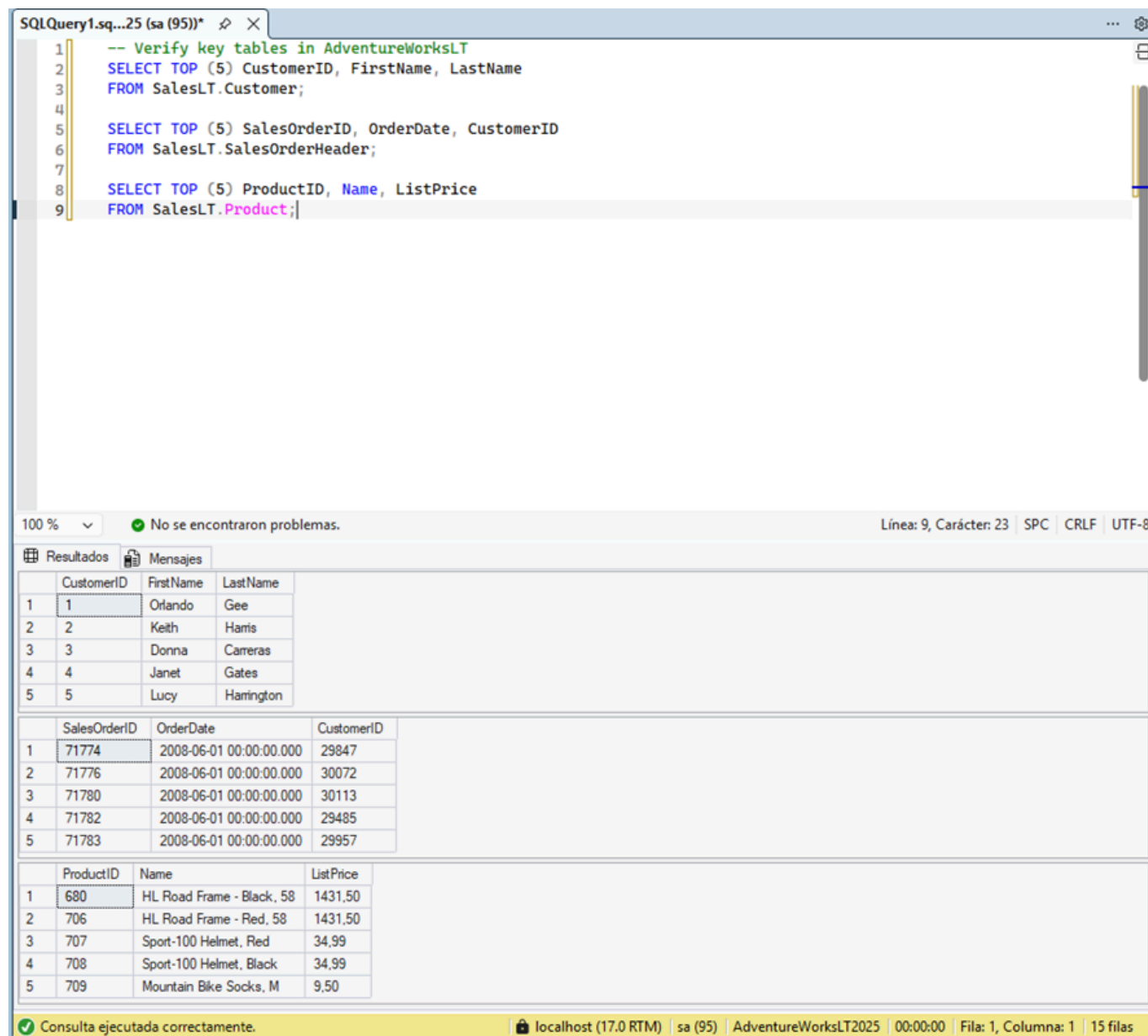


Conexión a AdventureWorksLT

Explicación: Asegúrese de que la base de datos de ejemplo AdventureWorksLT esté restaurada y accesible en su instancia de SQL. Verificaremos la conectividad y algunas tablas clave para confirmar que hay datos de ejemplo.

Resultado: Cada consulta debe devolver hasta cinco filas de datos. Si alguna no devuelve resultados, compruebe la restauración y permisos.



The screenshot shows a SQL query window titled "SQLQuery1.sql...25 (sa (95))". The query contains three SELECT statements to verify data in the AdventureWorksLT database:

```
1  -- Verify key tables in AdventureWorksLT
2  SELECT TOP (5) CustomerID, FirstName, LastName
3  FROM SalesLT.Customer;
4
5  SELECT TOP (5) SalesOrderID, OrderDate, CustomerID
6  FROM SalesLT.SalesOrderHeader;
7
8  SELECT TOP (5) ProductID, Name, ListPrice
9  FROM SalesLT.Product;
```

The Results pane displays the output of these queries:

	CustomerID	FirstName	LastName
1	1	Orlando	Gee
2	2	Keith	Harris
3	3	Donna	Carreras
4	4	Janet	Gates
5	5	Lucy	Harrington

	SalesOrderID	OrderDate	CustomerID
1	71774	2008-06-01 00:00:00.000	29847
2	71776	2008-06-01 00:00:00.000	30072
3	71780	2008-06-01 00:00:00.000	30113
4	71782	2008-06-01 00:00:00.000	29485
5	71783	2008-06-01 00:00:00.000	29957

	ProductID	Name	ListPrice
1	680	HL Road Frame - Black, 58	1431.50
2	706	HL Road Frame - Red, 58	1431.50
3	707	Sport-100 Helmet, Red	34.99
4	708	Sport-100 Helmet, Black	34.99
5	709	Mountain Bike Socks, M	9.50

The status bar at the bottom indicates: "Consulta ejecutada correctamente." and "localhost (17.0 RTM) | sa (95) | AdventureWorksLT2025 | 00:00:00 | Fila: 1, Columna: 1 | 15 filas".

Crear una vista para simplificar consultas

Explicación: Cree una vista que combine clientes y sus pedidos dentro del esquema SalesLT para ocultar la complejidad de los JOINS al código de aplicación.

Resultado: La consulta debe devolver hasta cinco filas con los pedidos más recientes y demostrar que la vista simplifica el acceso.

```
SQLQuery1.sql...25 (sa (95))*
1 CREATE OR ALTER VIEW SalesLT.vCustomerOrders AS
2 SELECT
3     c.CustomerID,
4     CONCAT(c.FirstName, ' ', c.LastName) AS CustomerName,
5     h.SalesOrderID,
6     h.OrderDate
7 FROM SalesLT.Customer c
8 INNER JOIN SalesLT.SalesOrderHeader h ON c.CustomerID = h.CustomerID;
```

100 % No se encontraron problemas. Línea: 8, Carácter: 32 SPC CRLF UTF-8

Mensajes

Los comandos se han completado correctamente.

Hora de finalización: 2026-09-22T10:49:01.8276586+02:00

100 % No se encontraron problemas. Línea: 4, Carácter: 1 TABULACIONES MIXTO UTF-8 with BOM

Consulta ejecutada correctamente. localhost (17.0 RTM) sa (95) AdventureWorksLT2025 00:00:00 Fila: 1, Columna: 1 0 filas

```
SQLQuery1.sql...25 (sa (95))*
1 SELECT TOP (5) *
2 FROM SalesLT.vCustomerOrders
3 ORDER BY OrderDate DESC;
```

100 % 2 0 Línea: 3, Carácter: 13 TABULACIONES CRLF UTF-8

Resultados Mensajes

	CustomerID	CustomerName	SalesOrderID	OrderDate
1	29847	David Hodgson	71774	2008-06-01 00:00:00.000
2	30072	Andrea Thomsen	71776	2008-06-01 00:00:00.000
3	30113	Raja Venugopal	71780	2008-06-01 00:00:00.000
4	29485	Catherine Abel	71782	2008-06-01 00:00:00.000
5	29957	Kevin Liu	71783	2008-06-01 00:00:00.000

Crear un procedimiento almacenado para procesar un pedido

Explicación: Encapsule una operación de negocio que agrega un artículo de pedido a un pedido existente y actualiza el subtotal del encabezado. El procedimiento valida productos y pedidos, inserta la línea y recalcula el subtotal dentro de una transacción.

Resultado: El procedimiento inserta la línea y actualiza el subtotal de forma atómica. Si hay errores, la transacción se revierte.

```
SQLQuery1.sql...25 (sa (95))  
24 BEGIN  
25     ROLLBACK TRANSACTION;  
26     THROW 50011, 'Invalid SalesOrderID specified.', 1;  
27 END  
28  
29 -- Insert line item (no discount)  
30 INSERT INTO SalesLT.SalesOrderDetail (SalesOrderID, OrderQty, ProductID, UnitPrice, UnitPriceDiscount)  
31 VALUES (@SalesOrderID, @Quantity, @ProductID, @UnitPrice, 0);  
32  
33 -- Update header subtotal based on current line totals  
34 UPDATE h  
35 SET SubTotal = d.SumLineTotal,  
36     ModifiedDate = SYSUTCDATETIME()  
37 FROM SalesLT.SalesOrderHeader h  
38 INNER JOIN (  
39     SELECT SalesOrderID, SUM(LineTotal) AS SumLineTotal  
40     FROM SalesLT.SalesOrderDetail  
41     WHERE SalesOrderID = @SalesOrderID  
42     GROUP BY SalesOrderID  
43 ) d ON d.SalesOrderID = h.SalesOrderID;  
44  
45 COMMIT TRANSACTION;  
46 END;
```

100 % No se encontraron problemas. Línea: 35, Carácter: 27, Columna: 29 SPC CRLF UTF-8

Mensajes

Los comandos se han completado correctamente.

Hora de finalización: 2026-09-22T10:55:34.1214947+02:00

SQLQuery1.sql...25 (sa (95))*

```
1  -- Add a line item to an existing order (choose a
2  DECLARE @SalesOrderID INT = (SELECT TOP 1 SalesOrderID
3                                FROM SalesLT.SalesOrderHeader
4                                ORDER BY SalesOrderID)
5  EXEC dbo.AddOrderLineItem @SalesOrderID = @SalesOrderID,
6                             @ProductID = 680,
7                             @Quantity = 1; -- adjust quantity
8
9  SELECT TOP (5) *
10 FROM SalesLT.SalesOrderDetail
11 WHERE SalesOrderID = @SalesOrderID
12 ORDER BY SalesOrderDetailID DESC;
13
14 SELECT SalesOrderID, SubTotal, TaxAmt, Freight, TotalDue
15 FROM SalesLT.SalesOrderHeader
16 WHERE SalesOrderID = @SalesOrderID;
```

100 % 1 0 Línea: 16, Carácter: 26 SPC C

Resultados Mensajes

	SalesOrderID	SalesOrderDetailID	OrderQty	ProductID	UnitPrice	UnitPriceDiscount
1	71946	113407	1	680	1431,50	0,00
2	71946	113406	1	916	31,584	0,00

	SalesOrderID	SubTotal	TaxAmt	Freight	TotalDue
1	71946	1463,084	3,1163	0,9738	1467,1741

Crear una función escalar para cálculos reutilizables

Explicación: Cree una función escalar que devuelva el valor total de un pedido sumando los line totals de SalesOrderDetail. Esto permite reutilizar la lógica en consultas.

Resultado: La función devuelve el total de cada pedido y se puede invocar desde consultas y joins.

```
SQLQuery1.sql...25 (sa (95))  
1 CREATE OR ALTER FUNCTION dbo.fnOrderTotal (@OrderID INT)  
2 RETURNS DECIMAL(18,2)  
3 AS  
4 BEGIN  
5     DECLARE @Total DECIMAL(18,2);  
6  
7     SELECT @Total = SUM(LineTotal)  
8     FROM SalesLT.SalesOrderDetail  
9     WHERE SalesOrderID = @OrderID;  
10  
11     RETURN ISNULL(@Total, 0.00);  
12 END;
```

100 % No se encontraron problemas. Línea: 12, Carácter: 10 SPC CRLF UTF-8

Mensajes

Los comandos se han completado correctamente.

Hora de finalización: 2026-09-22T11:18:06.6952608+02:00

SQLQuery1.sql...25 (sa (95))*

```
1 SELECT d.SalesOrderID, dbo.fnOrderTotal(d.SalesOrderID) AS OrderTotal
2 FROM SalesLT.SalesOrderDetail d
3 GROUP BY d.SalesOrderID
4 ORDER BY d.SalesOrderID DESC;
```

100 % 1 0

Resultados Mensajes

	SalesOrderID	OrderTotal
1	71946	1463.08
2	71938	74160.23
3	71936	79589.62
4	71935	5533.87
5	71923	96.11
6	71920	2527.13
7	71917	37.76
8	71915	1732.89
9	71902	59894.21
10	71899	1856.21
11	71898	53248.69
12	71897	10585.05
13	71895	221.26
14	71885	524.66
15	71867	858.90
16	71863	2777.14
17	71858	11528.84
18	71856	500.30

Crear una función con valor de tabla en línea (TVF)

Explicación: Cree una TVF que devuelva pedidos de un cliente dado. Las TVF son útiles para usar en SELECT y JOIN, y facilitan la reutilización de consultas parametrizadas.

Resultado: La TVF devuelve los pedidos del cliente y se puede aplicar en joins.

SQLQuery1.sql...25 (sa (95))*

```
1 CREATE OR ALTER FUNCTION dbo.GetCustomerOrders (@CustomerID INT)
2 RETURNS TABLE
3 AS
4 RETURN
5 (
6     SELECT
7         h.SalesOrderID,
8         h.OrderDate
9     FROM SalesLT.SalesOrderHeader h
10    WHERE h.CustomerID = @CustomerID
11 );
```

100 %

 No se encontraron problemas.

Mensajes

Los comandos se han completado correctamente.

Hora de finalización: 2026-09-22T11:19:05.8091879+02:00

SQLQuery1.sql...25 (sa (95))*

```
1 SELECT *
2 FROM dbo.GetCustomerOrders(29929)
3 ORDER BY OrderDate DESC;
```

100 % 2 0

Resultados Mensajes

	SalesOrderID	OrderDate
1	71902	2008-06-01 00:00:00.000

SQLQuery1.sql...25 (sa (95))*

```
1 SELECT CONCAT(c.FirstName, ' ', c.LastName) AS CustomerName, o.SalesOrderID, o.OrderDate
2 FROM SalesLT.Customer c
3 CROSS APPLY dbo.GetCustomerOrders(c.CustomerID) o
4 WHERE c.CustomerID = 29929;
```

100 % 3 0

Resultados Mensajes

	CustomerName	SalesOrderID	OrderDate
1	Jeffrey Kurtz	71902	2008-06-01 00:00:00.000

Crear un trigger para registrar cambios

Explicación: Añada un trigger que registre en una tabla de auditoría los cambios en los totales de los pedidos cuando cambien los detalles. Los triggers permiten aplicar reglas y capturar historial automáticamente.

Resultado: El SELECT devuelve las filas recientes de auditoría mostrando el OrderID afectado, el total anterior y el nuevo total con marca temporal.

```
SQLQuery1.sql...25 (sa (95))* X
1  -- Audit table
2  IF OBJECT_ID('dbo.OrderAudit') IS NULL
3  BEGIN
4      CREATE TABLE dbo.OrderAudit (
5          AuditID      INT IDENTITY(1,1) PRIMARY KEY,
6          OrderID       INT NOT NULL,
7          OldTotal      DECIMAL(18,2) NULL,
8          NewTotal      DECIMAL(18,2) NULL,
9          ChangedAt     DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
10     );
11 END
12 GO
13
14 -- Trigger on order details updates
15 CREATE OR ALTER TRIGGER SalesLT.trg_LogOrderTotalChange
16 ON SalesLT.SalesOrderDetail
17 AFTER INSERT, UPDATE
18 AS
19 BEGIN
20     SET NOCOUNT ON;
21
22     ;WITH AffectedOrders AS (
23         SELECT SalesOrderID FROM inserted
24     )
25
```

100 % No se encontraron problemas.

Mensajes

Los comandos se han completado correctamente.

Hora de finalización: 2026-09-22T11:20:30.1473724+02:00

```
SQLQuery1.sql...25 (sa (95))* X
1  -- Update an order detail to change the total
2  UPDATE d
3  SET OrderQty = OrderQty + 1
4  FROM SalesLT.SalesOrderDetail d
5  WHERE d.SalesOrderID = (SELECT TOP 1 SalesOrderID FROM SalesLT.SalesOrderHeader ORDER BY SalesOrderID DESC);
6
7  SELECT TOP (5) *
8  FROM dbo.OrderAudit
9  ORDER BY AuditID DESC;
```

100 % 2 0 ↑ ↓

Resultados

Mensajes

	AuditID	OrderID	OldTotal	NewTotal	ChangedAt
1	1	71946	1463.08	2926.17	2026-09-22 09:20:55.3495429